obraConnect

Documentação técnica completa

Arquitetura · Fluxos · Autenticação · Serviços · Avaliações · Segurança

Versão 1.0

1. Visão Geral da Aplicação

O obraConnect é uma plataforma de marketplace de serviços que conecta clientes a prestadores de serviços (ex: eletricistas, pedreiros, pintores). A aplicação é dividida em três camadas:

- Frontend — páginas HTML, CSS e JavaScript puro rodando no navegador do usuário.
- Backend — servidor Node.js com Express, responsável por toda a lógica de negócio e acesso ao banco de dados.
- Banco de dados — MySQL para persistir usuários, serviços e avaliações.

O fluxo básico de comunicação é: o usuário interage com uma página HTML → o JavaScript (via api.js) faz uma requisição HTTP para o backend → o backend processa, consulta o MySQL e retorna um JSON → o JavaScript atualiza a interface.

> *Todas as rotas que exigem autenticação recebem o token JWT no header Authorization: Bearer <token>. O token é armazenado no localStorage do navegador após o login.*

1.1. Estrutura de pastas

- frontend/ → index.html, style.css, /js/ (api.js, auth.js, servicos.js), /pages/
- backend/ → index.js, /config/ (database.js, upload.js), /middlewares/ (autenticacao.js), /routes/ (authRoutes.js, servicoRoutes.js, avaliacaoRoutes.js)
- _db/ → obraconnect_db.sql (schema completo do banco)

1.2. Tabela de todas as rotas da API

Método + Rota	Acesso	Descrição
`POST /api/auth/registro`	Público	Cadastrar novo usuário
`POST /api/auth/login`	Público	Autenticar e receber token JWT
`GET /api/auth/perfil`	🔒 JWT	Retornar dados do usuário logado
`PUT /api/auth/tornar-prestador`	🔒 JWT	Elevar tipo de conta para prestador
`GET /api/servicos`	Público	Listar todos os serviços ativos
`GET /api/servicos/:id`	Público	Detalhar um serviço específico
`GET /api/servicos/meus/servicos`	🔒 JWT	Listar serviços do usuário logado
`POST /api/servicos`	🔒 Prestador	Criar novo serviço (com upload de imagem)
`PUT /api/servicos/:id`	🔒 JWT	Editar serviço (somente o dono)

`PATCH /api/servicos/:id/desativar`	🔒 **JWT**	Desativar serviço (ativo = 0)
`PATCH /api/servicos/:id/ativar`	🔒 **JWT**	Reativar serviço (ativo = 1)
`DELETE /api/servicos/:id`	🔒 **JWT**	Deletar serviço e suas avaliações
`GET /api/avaliacoes/servico/:id`	**Público**	Listar avaliações + médias de um serviço
`GET /api/avaliacoes/meu-historico`	🔒 **JWT**	Listar avaliações feitas pelo usuário
`POST /api/avaliacoes`	🔒 **JWT**	Registrar nova avaliação
`DELETE /api/avaliacoes/:id`	🔒 **JWT**	Deletar avaliação (somente o dono)

2. Cadastro de Usuário

2.1. Rota de arquivos envolvida

1. Usuário preenche frontend/pages/registro.html
2. JavaScript chama registrarUsuario() em frontend/js/api.js
3. api.js dispara fetch POST para /api/auth/registro
4. backend/routes/authRoutes.js recebe a requisição
5. backend/config/database.js executa a query no MySQL

2.2. Campos obrigatórios

- nome_usuario — nome completo do usuário
- email — deve ter formato válido (validado por regex /^[^\s@]+@[^\s@]+\.[^\s@]+$/)
- login — identificador único de acesso (não pode repetir no banco)
- senha — mínimo 6 caracteres
- telefone — opcional, pode ser null

2.3. Validações no backend (em ordem)

6. Verifica se nome_usuario, email, senha e login foram enviados — erro 400 se faltar qualquer um.
7. Valida formato do email com regex — erro 400 se inválido.
8. Verifica se senha tem pelo menos 6 caracteres — erro 400 se menor.
9. Consulta o banco: SELECT * FROM oc__tb_usuario WHERE email = ? OR login = ? — se encontrar registro, retorna erro 409 (conflito).
10. Criptografa a senha com bcrypt (detalhado na seção 5).
11. Insere o novo registro com tipo_usuario = 'usuario' e data_cadastro = NOW().

2.4. O que é retornado

Status 201 com JSON contendo mensagem de sucesso, o id_usuario gerado pelo banco e o nome. O token NÃO é retornado no registro — o usuário precisa fazer login em seguida.

> *O tipo_usuario padrão ao criar uma conta é 'usuario'. Para postar serviços, o usuário precisa elevar seu tipo para 'prestador' via PUT /api/auth/tornar-prestador.*

3. Login e Autenticação

3.1. Rota de arquivos

12. Usuário preenche frontend/pages/login.html
13. JavaScript chama fazerLogin(loginOuEmail, senha) em frontend/js/api.js
14. api.js faz fetch POST /api/auth/login
15. authRoutes.js valida e gera o token JWT
16. Frontend salva o token no localStorage com a chave 'token'

3.2. Aceita login ou email

A query no banco usa OR: SELECT * FROM oc__tb_usuario WHERE login = ? OR email = ? — passando o mesmo valor para os dois parâmetros. Isso permite que o usuário entre com qualquer um dos dois campos.

3.3. Verificação de senha

A senha enviada nunca é comparada diretamente com o que está no banco. O banco guarda apenas o hash bcrypt. O backend usa bcrypt.compare(senhaEnviada, hashDoBanco), que internamente refaz o processo de hash e verifica se os resultados batem. Isso significa que a senha original nunca é descriptografada nem exposta.

3.4. O que é retornado após login bem-sucedido

- mensagem: 'Login realizado com sucesso!'
- token: o JWT assinado (válido pelo tempo definido em JWT_EXPIRY no .env)
- usuario: objeto com id, nome, email e tipo_usuario

3.5. Como o token é usado no frontend

Após o login, o frontend executa localStorage.setItem('token', resultado.token). A função genérica fazerRequisicao() em api.js lê esse token em toda requisição e o adiciona automaticamente ao header: Authorization: Bearer <token>. Rotas públicas funcionam sem o header; rotas protegidas retornam 401 se ele não estiver presente.

4. Como o Token JWT Funciona

4.1. Estrutura do JWT

Um JWT é uma string dividida em três partes separadas por ponto: header.payload.assinatura. As duas primeiras partes são base64url e legíveis; a terceira é o hash de verificação.

- Header: algoritmo de assinatura (HS256) e tipo do token.
- Payload: dados do usuário — id, nome, email e tipo_usuario.
- Assinatura: HMAC-SHA256 do header + payload usando o JWT_SECRET do .env. Qualquer alteração no payload invalida a assinatura.

4.2. Geração do token (authRoutes.js)

A função gerarToken(usuario) usa jwt.sign() com o payload {id, nome, email, tipo_usuario}, a chave secreta de process.env.JWT_SECRET e a expiração de process.env.JWT_EXPIRY (ex: '1h'). O token resultante é retornado para o frontend.

4.3. Validação do token — middleware verificarToken()

Cada requisição a uma rota protegida passa pelo middleware em backend/middlewares/autenticacao.js antes de chegar à lógica da rota. O middleware:

17. Lê req.headers.authorization e extrai a parte após 'Bearer '.
18. Se não houver token, retorna 401 com mensagem 'Token não fornecido'.
19. Chama jwt.verify(token, JWT_SECRET). Se a assinatura for inválida, retorna 403. Se o token estiver expirado, retorna 401 com 'Token expirado'.
20. Se válido, popula req.usuario com o payload decodificado (id, nome, email, tipo_usuario) e chama next() para prosseguir à rota.

4.4. Middleware verificarPrestador()

Usado nas rotas de criação de serviço. Verifica se req.usuario.tipo_usuario é 'prestador' ou 'admin'. Caso contrário, retorna 403. Esse middleware sempre vem depois de verificarToken().

4.5. Atualização do token ao virar prestador

Quando o usuário executa PUT /api/auth/tornar-prestador, o backend atualiza o campo tipo_usuario no banco para 'prestador' e imediatamente gera um novo token com os dados atualizados, devolvendo-o ao frontend. O frontend deve substituir o token antigo no localStorage pelo novo — sem isso, as próximas requisições ainda teriam o tipo 'usuario' no payload e seriam barradas pelo verificarPrestador().

> *O token contém dados em cache do momento do login. Se o banco for alterado diretamente, o token pode ficar desatualizado até expirar. Por isso, ao virar prestador, um novo token é gerado na mesma requisição.*

5. Criptografia de Senhas com bcrypt

5.1. O que é bcrypt

bcrypt é um algoritmo de hashing adaptativo projetado especificamente para senhas. Diferente de MD5 ou SHA-256, ele é intencionalmente lento e inclui um salt aleatório incorporado no resultado. A biblioteca usada é bcryptjs (implementação JavaScript pura, sem dependências nativas).

5.2. Como a senha é armazenada (no registro)

21. bcrypt.genSalt(10) gera um salt aleatório com fator de custo 10. Esse número determina quantas rodadas de hashing serão feitas (2^10 = 1024 iterações). Quanto maior, mais seguro e mais lento.
22. bcrypt.hash(senha, salt) combina a senha original com o salt e executa o algoritmo. O resultado é uma string de 60 caracteres que já inclui o salt embutido, o fator de custo e o hash final.
23. Apenas o hash final (60 chars) é salvo no banco. A senha original é descartada imediatamente após o hash.

5.3. Como a senha é verificada (no login)

bcrypt.compare(senhaTextoPlano, hashDoBanco) extrai o salt embutido no hash, recalcula o hash da senha enviada com aquele mesmo salt e compara. Se batem, retorna true. A senha original nunca é armazenada nem descriptografada em nenhum momento.

5.4. Por que isso é seguro

- Mesmo que o banco vaze, as senhas não são reversíveis — não existe operação inversa do bcrypt.
- Cada senha tem um salt único, então duas senhas iguais geram hashes diferentes. Ataques de rainbow table são ineficazes.
- O fator de custo pode ser aumentado no futuro sem invalidar senhas antigas, pois o custo fica embutido no hash.

6. Gerenciamento de Serviços

6.1. Criar um novo serviço

Pré-requisito

O usuário precisa ter tipo_usuario = 'prestador'. A rota POST /api/servicos aplica dois middlewares em sequência: verificarToken() e verificarPrestador(). Se o usuário for do tipo 'usuario', recebe 403 antes de chegar à lógica de criação.

Fluxo no frontend

24. Usuário preenche frontend/pages/cadastrar-servico.html com título, descrição, categoria e opcionalmente uma imagem.
25. O JS cria um objeto FormData (não JSON puro) e adiciona os campos. FormData é necessário porque há upload de arquivo binário.
26. Chama criarServico() em api.js, que faz fetch POST /api/servicos com Authorization no header, mas SEM Content-Type — o navegador define automaticamente multipart/form-data com o boundary correto quando usa FormData.

Fluxo no backend

27. O Multer (middleware de upload) intercede antes da lógica da rota e processa o arquivo da imagem.
28. Verifica se a extensão e o mimetype são permitidos: jpeg, jpg, png ou gif. Qualquer outro formato retorna erro.
29. Salva o arquivo em backend/uploads/ com nome único: fieldname-timestamp-random.extensao. Isso evita colisões de nome.
30. A rota valida se título e descrição foram enviados.
31. Monta a URL pública da imagem como protocolo://host/uploads/nome-arquivo.
32. Insere na tabela oc__tb_servico com id_usuario extraído do token JWT (não do body da requisição — impede que um usuário crie serviço em nome de outro).

Campos da tabela oc__tb_servico

- id — auto incremento
- id_usuario — FK para oc__tb_usuario (vem do token)
- titulo — texto obrigatório
- desc_servico — texto obrigatório
- id_categoria — FK opcional para tabela de categorias
- imagem_url — URL completa do arquivo salvo, ou null
- ativo — 1 (ativo) ou 0 (desativado), padrão 1
- data_cadastro — timestamp automático

6.2. Editar um serviço

PUT /api/servicos/:id — o backend primeiro busca o serviço pelo id e verifica se servicos[0].id_usuario === req.usuario.id. Se não for o dono, retorna 403. Se for, monta uma query UPDATE dinâmica: atualiza apenas os campos enviados, mantendo os valores originais para o que não foi informado. Se uma nova imagem for enviada, a URL é atualizada; caso contrário, a imagem anterior é preservada.

6.3. Ativar e desativar um serviço

A aplicação não deleta o serviço ao desativá-lo. Ela apenas muda o campo ativo no banco. A listagem pública usa a view oc__vw_servicos_ativos, que filtra apenas registros com ativo = 1. Isso significa que serviços desativados ficam invisíveis para o público, mas preservados no banco.

- PATCH /api/servicos/:id/desativar → UPDATE oc__tb_servico SET ativo = 0 WHERE id = ?
- PATCH /api/servicos/:id/ativar → UPDATE oc__tb_servico SET ativo = 1 WHERE id = ?

Em ambos, o backend verifica primeiro se o serviço existe e se o id_usuario do registro bate com o id do token. Sem essa verificação, qualquer usuário autenticado poderia desativar serviços alheios.

6.4. Deletar um serviço

DELETE /api/servicos/:id — além de verificar a propriedade do serviço, o backend executa duas queries em sequência:

33. DELETE FROM oc__tb_avaliacao WHERE id_servico = ? — remove as avaliações filhas primeiro para respeitar a chave estrangeira.
34. DELETE FROM oc__tb_servico WHERE id = ? — remove o serviço.

6.5. Views do banco usadas nos serviços

- oc__vw_servicos_ativos — retorna todos os serviços com ativo = 1, já com dados do prestador (join com oc__tb_usuario) e nota média calculada.
- oc__vw_detalhes_servico — retorna detalhes completos de um serviço específico, incluindo dados do prestador e médias de avaliação.

7. Sistema de Avaliações

7.1. Estrutura da avaliação

Diferente de uma nota única, o sistema usa quatro dimensões independentes, todas com escala de 1 a 5:

- nota_preco — avaliação do custo-benefício do serviço
- nota_tempo_execucao — avaliação do prazo de entrega
- nota_higiene — avaliação da organização e limpeza
- nota_educacao — avaliação do comportamento e comunicação do prestador
- comentario — campo de texto livre, opcional

7.2. Regras de negócio (validadas no backend)

35. Todas as quatro notas são obrigatórias — erro 400 se qualquer uma faltar.
36. Todas as notas devem estar entre 1 e 5 — validado com notas.some(n => n < 1 || n > 5).
37. O serviço deve existir — SELECT antes do INSERT.
38. O autor da avaliação não pode ser o dono do serviço — compara servicos[0].id_usuario com req.usuario.id, retorna 403 se iguais.
39. Cada usuário pode avaliar um serviço apenas uma vez — SELECT antes do INSERT verifica duplicidade, retorna 400 se já avaliou.

7.3. Cálculo de médias

As médias NÃO são armazenadas no banco. Elas são calculadas em tempo real pelo backend toda vez que a rota GET /api/avaliacoes/servico/:id é chamada. O backend faz reduce() sobre o array de avaliações para somar cada dimensão, divide pelo total e usa toFixed(1) para arredondar a uma casa decimal. A média geral é calculada somando todas as quatro dimensões e dividindo por (total * 4).

7.4. Deletar avaliação

DELETE /api/avaliacoes/:id — verifica se a avaliação existe e se o id_usuario dela bate com o id do token. Sem essa verificação, um usuário poderia deletar avaliações de outros. Retorna 403 em caso de tentativa indevida.

8. Upload de Imagens (Multer)

8.1. Configuração em backend/config/upload.js

O Multer é configurado com três aspectos principais: storage (destino e nome do arquivo), fileFilter (validação de tipo) e limits (tamanho máximo).

8.2. Destino e nome do arquivo

- Destino: backend/uploads/ — criado automaticamente se não existir (fs.mkdirSync com recursive: true).
- Nome único: `fieldname + '-' + Date.now() + '-' + Math.round(Math.random() * 1e9) + extensão original`
- O timestamp garante unicidade temporal; o número aleatório de 9 dígitos previne colisões em uploads simultâneos.

8.3. Validação de tipo

O fileFilter verifica dois aspectos: a extensão do nome do arquivo (regex /jpeg|jpg|png|gif/) e o mimetype real do arquivo (image/jpeg, image/png, image/gif). Ambos precisam passar — um arquivo renomeado com extensão errada é rejeitado.

8.4. Limite de tamanho

O tamanho máximo é lido de process.env.MAX_FILE_SIZE ou usa 5MB como padrão (5 * 1024 * 1024 bytes). Se o arquivo exceder o limite, o Multer retorna um erro antes mesmo de salvar qualquer coisa.

8.5. Acesso público às imagens

O backend serve a pasta uploads/ como rota estática via express.static: app.use('/uploads', express.static(path.join(__dirname, 'uploads'))). Assim, uma imagem salva como imagem-123456789.jpg fica acessível publicamente em http://localhost:3001/uploads/imagem-123456789.jpg sem autenticação.

9. Banco de Dados MySQL

9.1. Conexão (backend/config/database.js)

A conexão usa mysql2/promise, que permite async/await. É criado um pool de conexões com limite de 10 conexões simultâneas (connectionLimit: 10). Ao iniciar, o backend testa imediatamente a conexão com pool.getConnection(). Se falhar, process.exit(1) encerra o servidor, evitando que ele suba sem banco.

9.2. Tabelas principais

- oc__tb_usuario — id, nome_usuario, email, senha (hash), login, telefone, tipo_usuario, data_cadastro
- oc__tb_servico — id, id_usuario (FK), titulo, desc_servico, id_categoria (FK), imagem_url, ativo, data_cadastro
- oc__tb_avaliacao — id, id_servico (FK), id_usuario (FK), nota_preco, nota_tempo_execucao, nota_higiene, nota_educacao, comentario, data_avaliacao

9.3. Views usadas pela API

- oc__vw_servicos_ativos — serviços com ativo = 1, com join no usuário e média de notas
- oc__vw_detalhes_servico — detalhes completos de um serviço com dados do prestador
- oc__vw_avaliacoes_do_servico — avaliações com nome do avaliador
- oc__vw_historico_de_avaliacoes — avaliações feitas por um usuário com nome do serviço

9.4. Prevenção de SQL injection

Todas as queries usam placeholders ? com parâmetros separados: banco.query('SELECT * FROM tabela WHERE id = ?', [id]). O driver mysql2 nunca interpola strings diretamente no SQL — os parâmetros são enviados separadamente ao MySQL, que os escapa internamente. Isso torna injeção de SQL estruturalmente impossível nas queries existentes.

10. Segurança Geral

10.1. Resumo das camadas de segurança

Camada	Mecanismo
Senhas	Hash bcrypt com salt 10 — irreversível, único por senha
Autenticação	JWT assinado com HS256 e JWT_SECRET do .env — inforjável sem a chave
Autorização	Middlewares verificarToken() e verificarPrestador() em toda rota sensível
Propriedade	Toda operação de edição/deleção verifica se id_usuario do registro = id do token
SQL Injection	Placeholders ? em todas as queries — parâmetros nunca interpolados
Uploads	Validação de extensão E mimetype; limite de 5MB; nomes únicos com timestamp
CORS	Configurado em app.use(cors()) — deve ser restrito para domínio do frontend em produção
Segredos	.env nunca commitado no Git — chaves JWT e credenciais do banco fora do código

10.2. Tipos de usuário e permissões

- usuario — pode fazer login, ver serviços, avaliar serviços de outros, ver próprio perfil.
- prestador — todos os direitos de 'usuario' + pode criar, editar, ativar/desativar e deletar próprios serviços.
- admin — todos os direitos de 'prestador' + bypass do verificarPrestador().

10.3. Boas práticas para produção

- Restringir CORS: origin: 'https://seu-dominio.com' em vez de '*'.
- Usar HTTPS para todas as requisições.
- Armazenar JWT_SECRET e credenciais do banco em variáveis de ambiente do servidor (AWS Secrets Manager, Azure Key Vault, Docker secrets), nunca no código.
- Aumentar o fator de custo do bcrypt (de 10 para 12 ou 14) conforme o hardware do servidor evoluir.
- Monitorar respostas 4xx e 5xx para detectar tentativas de ataque.
- Fazer backup do banco antes de qualquer migração ou deploy.

11. Configuração e Setup

11.1. Variáveis de ambiente (.env)

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=sua_senha
DB_NAME=obraconnect_db
DB_PORT=3306
JWT_SECRET=chave_longa_e_aleatoria
JWT_EXPIRY=1h
PORT=3001
NODE_ENV=development
MAX_FILE_SIZE=5242880    # 5MB em bytes (opcional)
```

11.2. Dependências Node.js do backend

- express — framework HTTP, cria e gerencia as rotas REST
- mysql2 — driver MySQL com suporte nativo a promises e placeholders
- dotenv — carrega variáveis do .env para process.env
- bcryptjs — hashing de senhas, implementação JS pura sem dependências nativas
- jsonwebtoken — gera e verifica tokens JWT
- cors — middleware que adiciona headers de CORS às respostas
- multer — processa uploads de arquivos multipart/form-data
- nodemon (dev) — reinicia o servidor automaticamente ao salvar arquivos

11.3. Ordem de inicialização do backend (index.js)

40. dotenv.config() — carrega variáveis de ambiente
41. require('./config/database') — cria pool e testa conexão MySQL
42. app.use(cors(...)) — configura CORS globalmente
43. app.use(express.json()) — habilita parse de corpo JSON
44. app.use('/uploads', express.static(...)) — serve arquivos de upload
45. app.use('/api/auth', authRoutes) — monta rotas de autenticação
46. app.use('/api/servicos', servicoRoutes) — monta rotas de serviços
47. app.use('/api/avaliacoes', avaliacaoRoutes) — monta rotas de avaliações
48. Middleware 404 — captura rotas não encontradas
49. Middleware de erro global — captura erros de qualquer rota
50. app.listen(PORT) — inicia o servidor

Documentação gerada automaticamente com base no código-fonte do projeto obraConnect.